

W tym zadaniu należy zaimplementować parser, składnię abstrakcyjną oraz ewaluator języka zapytań PicoQL.

Język PicoQL to prosty język zapytań pozwalający na wyszukiwanie i transformację danych tabelarycznych (można transformować tylko pojedynczą tabelę). Język składa się z ciągu poleceń, każde przekształcające tabelę w nową tabelę po kolei, od lewej do prawej.

Polecenia

`FILTER e;`

gdzie `e` to wyrażenie języka. Pozostaw w tabeli tylko te wiersze, dla których wartością wyrażenia `e` jest prawda.

`DROP COLUMN c;`

gdzie `c` to nazwa kolumny. Usuń z tabeli kolumnę `c`.

`ADD COLUMN c WITH e;`

gdzie `c` to nazwa kolumny, a `e` to wyrażenie w języku. Dodaj do tabeli kolumnę o nazwie `c`. W każdym wierszu wypełniaj nową komórkę wartością wyrażenia `e`.

`LIMIT i;`

gdzie `i` to liczba naturalna. Pozostaw w tabeli tylko `i` wierszy (lub mniej, jeśli wejściowa tabela ma ich mniej).

`ROW NUMBER c;`

gdzie `c` to nazwa kolumny. Dodaj do tabeli nową kolumnę o nazwie `c`, która dla każdego wiersza zawiera numer tego wiersza. Wiersze zaczynamy liczyć od 0.

Wyrażenia

W poleceniach `ADD COLUMN` i `FILTER` wyrażenia `e` mogą zawierać zmienne wolne o nazwach odpowiadających nazwom kolumn w tabeli.

Podczas ewaluacji wyrażenia dla każdego wiersza, wartościami tych zmiennych są wartości w danym wierszu znajdujące się w odpowiadających kolumnach.

Założ, że nazwy kolumn są prawidłowymi identyfikatorami języka.

Przykład

Przykładowo, polecenie:

`ADD COLUMN suma WITH x + y;`

zastosowane do tabeli:

x,y
10,20
11,5

powinno wyprodukować tabelę:

```
x,y,suma
10,20,30
11,5,16
```

Kolejność kolumn nie ma znaczenia.

Możesz założyć, że `c` w poleceniach `ADD COLUMN c` oraz `ROW NUMBER c` nie występuje w tabeli wejściowej.

Reprezentacja tabeli

Tabelę reprezentujemy jako typ:

```
type column = string
type row = Eval.value list
type table = column list * row list
```

Zadanie

Należy zdefiniować składnię abstrakcyjną języka w typie:

```
type query = ...
```

oraz ewaluator:

```
let eval (q : query) (t : table) : table = ...
```

Parser

W parserze zdefiniowano już składnię konkretną, ale nie jest ona jeszcze tłumaczona do składni abstrakcyjnej.

Należy uzupełnić to tłumaczenie w pliku `Parser.mly`.

Uruchomienie

Po implementacji należy zmienić nazwę pliku:

```
Query.ml → solution.ml
```

Następnie zainstalować bibliotekę `csv`:

```
opam install csv
```

Program kompiluje się do narzędzia, które jako argumenty bierze zapytanie i ścieżkę do pliku CSV, a wynik wypisuje w formacie CSV.

Przykłady użycia

```
cat mountains.csv
```

mountain,height,year,climbers
Everest,8848,1953,"Edmund Hillary, Tenzing Norgay"
K2,8611,1954,"Achille Compagnoni, Lino Lacedelli"
Kangchenjunga,8586,1955,"George Band, Joe Brown"

dune exec picoql "DROP COLUMN climbers; FILTER year >= 1955; LIMIT 3;" mountains.csv

mountain,height,year
Kangchenjunga,8586,1955
Lhotse,8516,1956
Makalu,8485,1955

dune exec picoql "DROP COLUMN climbers; DROP COLUMN year; ADD COLUMN height_in_feet WITH
(height * 3280839) / 1000000; LIMIT 3;" mountains.csv

mountain,height,height_in_feet
Everest,8848,29028
K2,8611,28251
Kangchenjunga,8586,28169